

# Android 第五章第二节教学讲义（中级控件进阶版）

各位同学，今天咱们继续深入学习 Android 中级控件的核心知识。上节课咱们认识了一些基础的中级控件和图形工具，这节课重点讲解状态变化相关的控件（比如复选框、开关按钮）、文本输入的高级用法，以及对话框的更多细节。内容贴近实际开发，咱们用 "场景 + 代码" 的方式来讲，保证你学完就能用！

## 一、状态列表图形：让控件 "聪明变脸"

平时点击按钮时，按钮会从 "正常色" 变成 "按压色"；选中复选框时，方框会变成 "打钩状态"—— 这些都是因为用了**状态列表图形（StateListDrawable）**。

简单说：状态列表图形是一种 XML 文件，能根据控件的不同状态（比如正常、按压、选中）显示不同的样式。就像给控件准备了多套 "衣服"，它会根据自己的状态穿对应的衣服。

### 实战：给按钮做 "按压变色" 效果

#### 步骤 1：创建状态列表 XML（res/drawable/btn\_state.xml）

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- 1. 按压状态（手指按下去时） -->
  <item android:state_pressed="true">
    <!-- 按压时显示的样式：用Shape画一个深红色圆角矩形 -->
    <shape android:shape="rectangle">
      <solid android:color="#F44336" /> <!-- 红色 -->
      <corners android:radius="8dp" /> <!-- 圆角 -->
    </shape>
  </item>
  <!-- 2. 正常状态（默认显示） -->
  <item>
    <!-- 正常时显示的样式：浅蓝色圆角矩形 -->
    <shape android:shape="rectangle">
```

```
<solid android:color="#2196F3" /> <!-- 蓝色 -->
<corners android:radius="8dp" />
</shape>
</item>
</selector>
```

**注意：**状态列表中，**具体状态要写在前面**，默认状态写在最后（系统会从上到下匹配状态）。

## 步骤 2：给按钮引用这个状态列表

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="点我变色"
    android:background="@drawable/btn_state" /> <!-- 引用状态列表 -->
```

效果：按钮平时是蓝色，手指按下去瞬间变成红色，松开又变回蓝色 —— 交互感一下就上来了！

## 二、选择类控件：让用户做 "选择题"

选择类控件是 APP 中最常用的交互元素，比如注册时选性别、设置里开功能，都需要它们。咱们重点讲 3 个：CheckBox（复选框）、Switch（开关）、RadioButton（单选按钮）。

### 1. CheckBox（复选框）：可多选的 "小方框"

场景：选择兴趣爱好（可以选多个，比如 "读书" 和 "运动" 都选）。

#### 布局文件（activity\_main.xml）

```
<!-- 复选框1：读书 -->
<CheckBox
    android:id="@+id/cb_book"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="读书"
    android:textSize="16sp" />
```

```
<!-- 复选框2： 运动 -->
<CheckBox
    android:id="@+id/cb_sport"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="运动"
    android:textSize="16sp" />
```

## 代码中监听选中状态

```
// 找到复选框
CheckBox cbBook = findViewById(R.id.cb_book);
CheckBox cbSport = findViewById(R.id.cb_sport);
// 设置选中变化监听器
cbBook.setOnCheckedChangeListener(new CompoundButton.OnCheckedChangeListener()
{
    @Override
    public void onCheckedChanged(CompoundButton buttonView, boolean isChecked) {
        // isChecked为true：选中；false：未选中
        if (isChecked) {
            Toast.makeText(MainActivity.this, "选中了读书", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(MainActivity.this, "取消了读书", Toast.LENGTH_SHORT).show();
        }
    }
});
// 运动复选框同理（可以复用监听器，也可以单独写）
cbSport.setOnCheckedChangeListener((buttonView, isChecked) -> {
    if (isChecked) {
        Toast.makeText(MainActivity.this, "选中了运动", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(MainActivity.this, "取消了运动", Toast.LENGTH_SHORT).show();
    }
});
```

## 2. Switch（开关按钮）：一键开启 / 关闭

场景：设置中的 "夜间模式"、"消息通知" 等功能（开或关，二选一）。

## 布局文件

```
<Switch
    android:id="@+id/switch_notify"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="消息通知" <!-- 开关旁边的文字 -->
    android:checked="true" /> <!-- 默认是否开启（true为开） -->
```

## 代码中监听状态

```
Switch switchNotify = findViewById(R.id.switch_notify);
switchNotify.setOnCheckedChangeListener((buttonView, isChecked) -> {
    if (isChecked) {
        // 开关打开：开启消息通知
        Toast.makeText(MainActivity.this, "消息通知已开启", Toast.LENGTH_SHORT).show();
    } else {
        // 开关关闭：关闭消息通知
        Toast.makeText(MainActivity.this, "消息通知已关闭", Toast.LENGTH_SHORT).show();
    }
});
```

## 3. RadioButton（单选按钮）：只能选一个的"小圆圈"

场景：选择性别（男 / 女，只能选一个），需要配合RadioGroup使用（保证单选效果）。

## 布局文件

```
<!-- 单选按钮组：垂直排列 -->
<RadioGroup
    android:id="@+id/rg_gender"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <!-- 单选按钮1：男 -->
```

```
<RadioButton
    android:id="@+id/rb_male"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="男" />
<!-- 单选按钮2: 女 -->
<RadioButton
    android:id="@+id/rb_female"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="女" />
</RadioGroup>
```

## 代码中获取选中项

```
RadioGroup rgGender = findViewById(R.id.rg_gender);
// 监听选中变化
rgGender.setOnCheckedChangeListener((group, checkedId) -> {
    // checkedId是选中的RadioButton的ID
    if (checkedId == R.id.rb_male) {
        Toast.makeText(MainActivity.this, "选中了男", Toast.LENGTH_SHORT).show();
    } else if (checkedId == R.id.rb_female) {
        Toast.makeText(MainActivity.this, "选中了女", Toast.LENGTH_SHORT).show();
    }
});
```

## 三、EditText（文本输入框）：高级用法 + 监听器

EditText 是输入文字的控件，除了基础的输入功能，还能通过监听器实时获取输入内容，或判断输入焦点（比如用户是否正在输入）。

### 1. EditText 高级属性（布局中）

```
<EditText
    android:id="@+id/et_username"
    android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
android:hint="请输入用户名" <!-- 提示文字 -->
android:inputType="textPersonName" <!-- 输入类型：人名（软键盘会优化） -->
android:maxLines="1" <!-- 最多1行（不换行） -->
android:maxLength="10" <!-- 最多输入10个字符 -->
android:padding="10dp" /> <!-- 内边距（文字不贴边） -->
```

## 2. 焦点变更监听器：判断用户是否在输入

场景：当用户点击输入框（获取焦点）时，提示 "请输入正确格式"；当用户点击其他地方（失去焦点）时，验证输入内容。

```
EditText etUsername = findViewById(R.id.et_username);
// 设置焦点监听器
etUsername.setOnFocusChangeListener((v, hasFocus) -> {
    if (hasFocus) {
        // 获得焦点（用户开始输入）
        Toast.makeText(MainActivity.this, "请输入用户名（最多10字）", Toast.LENGTH_SHORT).show();
    } else {
        // 失去焦点（用户输入完毕）
        String username = etUsername.getText().toString();
        if (username.isEmpty()) {
            Toast.makeText(MainActivity.this, "用户名不能为空", Toast.LENGTH_SHORT).show();
        }
    }
});
```

## 3. 文本变化监听器：实时监听输入内容

场景：注册时实时显示 "已输入 3/10 字符"，或实时验证手机号格式。

```
EditText etPhone = findViewById(R.id.et_phone);
// 设置文本变化监听器
etPhone.addTextChangedListener(new TextWatcher() {
    @Override
    public void beforeTextChanged(CharSequence s, int start, int count, int after) {
        // 文字变化前调用（暂时用不到）
    }
});
```

```

}
@Override
public void onTextChanged(CharSequence s, int start, int before, int count) {
    // 文字正在变化时调用 (s是当前输入的内容)
    String phone = s.toString();
    // 显示已输入的长度
    Toast.makeText(MainActivity.this, "已输入" + phone.length() + "位", Toast.LENGTH_SHORT).show();
}
@Override
public void afterTextChanged(Editable s) {
    // 文字变化后调用 (比如验证格式)
    String phone = s.toString();
    if (phone.length() == 11) {
        // 假设11位是手机号, 验证格式
        if (phone.startsWith("13") || phone.startsWith("15") || phone.startsWith("18")) {
            Toast.makeText(MainActivity.this, "手机号格式正确", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(MainActivity.this, "手机号格式错误", Toast.LENGTH_SHORT).show();
        }
    }
}
};

```

## 四、AlertDialog（对话框）：不止是“确认”和“取消”

上节课咱们简单认识了对话框，这节课讲更多实用样式，比如带列表的对话框、带自定义布局的对话框。

### 1. 带列表的对话框（选择列表）

场景：让用户从多个选项中选一个（比如选择“排序方式”）。

```

// 选项列表
String[] options = {"按时间排序", "按名称排序", "按大小排序"};
new AlertDialog.Builder(this)
    .setTitle("请选择排序方式") // 标题
    .setItems(options, (dialog, which) -> {

```

```
// which是选中的选项索引 (0开始)
Toast.makeText(MainActivity.this, "你选了: " + options[which], Toast.LENGTH_SHORT).
show();
})
.setNegativeButton("取消", null) // 取消按钮 (null表示点击后只关闭对话框)
.show();
```

## 2. 带自定义布局的对话框（灵活展示）

场景：弹出一个“登录框”（包含账号、密码输入框），这种复杂布局需要自定义。

### 步骤 1：创建自定义布局（res/layout/dialog\_login.xml）

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="15dp">
    <EditText
        android:id="@+id/et_dialog_username"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="账号" />
    <EditText
        android:id="@+id/et_dialog_password"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="密码"
        android:inputType="textPassword"
        android:layout_marginTop="10dp" />
</LinearLayout>
```

### 步骤 2：在代码中加载自定义布局

```
// 加载自定义布局
View loginView = LayoutInflater.from(this).inflate(R.layout.dialog_login, null);
// 找到布局中的输入框
```



```
EditText etDialogUser = loginView.findViewById(R.id.et_dialog_username);
EditText etDialogPwd = loginView.findViewById(R.id.et_dialog_password);
new AlertDialog.Builder(this)
    .setTitle("请登录")
    .setView(loginView) // 设置自定义布局
    .setPositiveButton("登录", (dialog, which) -> {
        // 获取输入的账号密码
        String user = etDialogUser.getText().toString();
        String pwd = etDialogPwd.getText().toString();
        Toast.makeText(MainActivity.this, "账号: " + user + ", 密码: " + pwd, Toast.LENGTH_
SHORT).show();
    })
    .setNegativeButton("取消", null)
    .show();
```

## 总结

这节课咱们学了 8 个核心知识点，都是日常开发高频使用的：

1. **状态列表图形**：用 selector 标签定义控件在不同状态（按压、选中）的样式，让交互更生动；
2. **CheckBox**：复选框，可多选，用 `setOnCheckedChangeListener` 监听状态；
3. **Switch**：开关按钮，适合开启 / 关闭功能，同样用 `setOnCheckedChangeListener`；
4. **RadioButton**：单选按钮，需放在 `RadioGroup` 中，通过组的监听器获取选中项；
5. **EditText**：高级属性（限制输入类型、长度）让输入更规范；
6. **焦点变更监听器**： `setOnFocusChangeListener` 判断输入框是否被激活；
7. **文本变化监听器**： `addTextChangedListener` 实时监听输入内容，适合动态验证；
8. **AlertDialog**：除了基础确认框，还能做列表对话框、自定义布局对话框。

这些控件和监听器是构建 APP 交互的核心，建议课后做一个 "注册页面" 练习：用 `RadioButton` 选性别，`CheckBox` 选爱好，`EditText` 输入账号密码，并用文本监听器实时验证，最后用对话框提示注册结果。练完这一套，你对中级控件的理解会更深刻！有问题随时问～

（注：文档部分内容可能由 AI 生成）